Aufgabenübersicht

1. Gestenerkennung im Alltag KI-A3.1.1

Wir alle benutzen Gestenerkennung im Alltag in verschiedenen Situationen. Dabei verwenden wir häufig nur unsere Hände, z.B. um zu kommunizieren, wo Unterhaltungen nicht möglich sind. Oft werden Gesten auch digital verarbeitet, z.B. zur Steuerung von Computern.

a) Überlege, wo dir Gestenerkennung im Alltag begegnet?

Gesten werden häufig zur Steuerung von Computern verwendet. Mit der sogenannten „Pinch-Zoom“ [1] -Geste, bei der Daumen und Zeigefinger einer Hand zusammen- oder auseinanderbewegt werden, womit Dinge auf einem Bildschirm verkleinert oder vergrößert werden können.

b) Diskutiere anhand dieses Beispiels, was die Vorteile und Nachteile von Gesten zur Steuerung bestimmter Anwendungen sein könnten.

c) Fallen dir weitere Vor- oder Nachteile von Gestenerkennung ein?

2. Ein Zeichenprogramm erstellen KI-A3.1.2

Es gibt mehrere Formen von Gesten z.B. bei Mustern zum Entsperren eines Mobiltelefons. Einige Gesten bestehen aus nur einem Strich, manche aus mehreren Strichen. Unter sogenannten **Einstrichgesten** versteht man Gesten, die an einem Stück, ohne abzusetzen, erzeugt werden. **Mehrstrichgesten** bestehen aus mehreren Strichen.

2.1 Zeichenprogramm selbst gemacht mit Snap!

Um später eine Gestenerkennung programmieren können, benötigst du zuerst ein Programm, mit dem du **Einstrichgesten aufzeichnen** kannst.

Dieses Zeichenprogramm soll:

- starten, sobald die Maus auf der Bühne[2] gedrückt wird.
- mögliche vorherige Zeichnungen von der Bühne wischen.
- die Bewegung der Maus auf der Bühne zeichnen, solange die Maus gedrückt ist.
- die Bewegung der Maus nicht mehr zeichnen, sobald die Maus nicht mehr gedrückt ist.

a) Überlege zunächst selbst oder mit anderen, wie du ein Zeichenprogramm programmieren kannst, das die oben genannten Anforderungen erfüllt.

Tipps zum Programmieren des Zeichenprogramms finden sich in KI-A3.1.2 auf S.2 Hinweis: Du kannst dein Zeichenprogramm unter dieser URL programmieren: https://tinyurl.com/GrosseGesten

[1] Von englisch „pinch“ = kneifen

[2] Als Bühne bezeichnet man das Fenster im oberen rechten Bereich von Snap!, in dem die Programme abgespielt werden.

Eine Entwicklung von OFFIS e.V. in Kooperation mit der Universität Oldenburg im Auftrag der Wissensfabrik – Unternehmen für Deutschland e.V.

2.2 Testen des Zeichenprogramms

a) Überlege dir einige Einstrichgesten und versuche, sie mit deinem Zeichenprogramm zu erstellen.

b) Versuche nun Mehrstrichgesten. Inwiefern funktionieren diese?

3. Strich in Bewegung bringen KI-B3.1.3

Da du das Zeichenprogramm selbst programmierst, kannst du es tun lassen, was du willst. In diesem Abschnitt geht es darum, die Zeichnungen in Bewegungen zu bringen.

a) Teste den „animiere“ Block mit einfachen Beispielen. Wie du den animiere Block findest oder in deine Programmierung einbindest findest du auf dem Arbeitsmaterial KI-B3.1.3 (Seite 1).

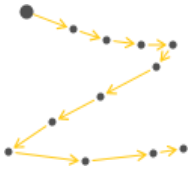
b) Überlege dir zwei Zeichen, die mit Einstrichgesten erzeugt werden können, und dazu passende Animationsideen.

Zeichen	Beispiel: 		
Animations-idee	Herzschlag erzeugen. Zeichnung mehrmals verkleinern und vergrößern		

c) Setze deine Ideen in deinem Snap! Projekt um.

4. Punkt vor Strich: Pfade Erzeugen KI-B3.1.4

Nun sollen die Einstrichgesten als Bewegung umgesetzt werden, als Abfolge von Punkten, die ein Malstift durchläuft, um etwas zu zeichnen. Eine Solche Abfolge von Punkten nennt sich „Pfad“.



Beispiel für einen “Zickzack”-Pfad, der ein Z-förmiges Zeichen malt.

Weiter Beispiele finden sich auf dem Arbeitsmaterial KI-B3.1.4

a) Zeichne zwei eigene weitere Beispiele für einen Pfad

Eine Entwicklung von OFFIS e.V. in Kooperation mit der Universität Oldenburg
im Auftrag der Wissensfabrik – Unternehmen für Deutschland e.V.

4.1 Einen Pfad aufzeichnen

Um in Snap! Einen Pfad aufzuzeichnen bzw. abzuspeichern wird eine Variable benötigt.

b) Gibt deiner Variable für den Pfad einen aussagekräftigen Namen, wie beispielsweise „Zeichen“.

In der neuen Variable soll nun jedes Mal, wenn dein Programm ein Bild malt, die Abfolge der durchlaufenen Punkte aufgezeichnet werden. Dafür benötigst du die folgenden Snap! Blöcke:



Diese Anweisung initialisiert deine Variable als Liste.



Diese Anweisung speichert einen Punkt deines Pfades als Liste mit zwei Werten der x- Position und der y-Position in deiner vorher initialisierten Liste.

c) Füge nun die beiden obigen Anweisungen in dein Malprogramm ein, damit die Liste:

- am Anfang einer neuen Geste auf eine leere Liste gesetzt wird.
- die aktuelle x- und y-Position nach jedem Positionswechsel zur Liste hinzugefügt wird.

4.2 Einen Pfad Abspielen

Nachdem dein Pfad aufgezeichnet wurde, geht es nun darum, diesen auch abzuspielen. Hierfür muss die Figur zu den aufgezeichneten Punkten bewegt werden. Dafür kann die Figur an einem bestimmten Punkt mit einer x-Position und einer y-Position wiedergegeben werden.





Zusätzlich gibt es die Möglichkeit, alle Elemente in einer Liste nacheinander abzuspielen. Dies geschieht mit Hilfe von bestimmten Blöcken, den sogenannten Schleifen. Eine Schleife ist in der Programmierung wie ein automatisierter Wiederholungsprozess, der eine bestimmte Aufgabe oder eine Gruppe von Aufgaben wiederholt ausführt, bis eine bestimmte Bedingung erfüllt oder eine bestimmte Anzahl Wiederholungen erreicht ist. In unserem Beispiel dient die Schleife dazu, jedes Element aus deiner Liste auszuführen bzw. aufzuzeichnen.

d) Male ein Zeichen und lasse es danach abspielen. Wenn du alles richtig gemacht hast, dann gibt das Programm dein Zeichen exakt so wieder, wie du es zuvor gemalt hast.

Die folgenden Snap! Blöcke können dir dabei helfen:



Der Block hat eine eingebaute Variable namens “Element”. Von dieser Variablen kannst du ganz einfach Kopien “abziehen” und in anderen Blöcken innerhalb der Schleife verwenden.





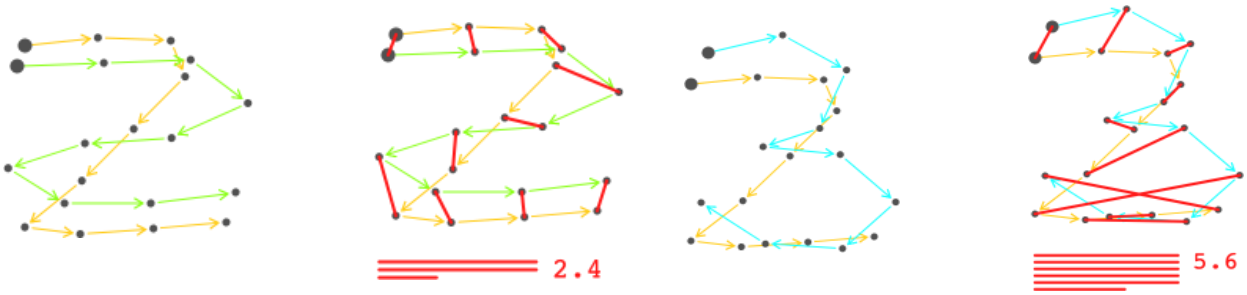


Eine Entwicklung von OFFIS e.V. in Kooperation mit der Universität Oldenburg
im Auftrag der Wissensfabrik – Unternehmen für Deutschland e.V.

5. Pfade vergleichen

Nachdem du nun einen Pfad aufzeichnen, abspeichern und abspielen kannst wollen wir nun den Unterschied bzw. die Ähnlichkeit von verschiedenen Zeichen bzw. zwei Pfaden bestimmen.

Zwei Pfade können anhand der Anzahl ihrer Punkte vergleichen werden:



Zwei Pfade können nur vergleichen werden, wenn diese die gleiche Anzahl an Punkten haben. Die Summe der Entfernungen jedes Punktepaares kann dann als “Unterschied” zwischen beiden Pfaden dienen. Auf diese Weise kann man vergleichen, ob der Unterschied zu einem dritten Pfad kleiner oder größer ist.

5.1 Die Anzahl der Punkte verändern

Damit, wie oben beschrieben, zwei Zeichen Punkt für Punkt miteinander verglichen werden können, müssen beide Pfade gleich viele Punkte haben. Mit dem „resample“ Snap!-Block kannst du bestimmen, wie viele Punkte ein Pfad bzw. Zeichen haben soll.





Zudem kannst du die von dir aufgezeichneten einzelnen Punkte eines Zeichens sichtbar machen mit:



Mit dem “resample” Block kannst du nun die Anzahl der Punkte genau bestimmen und gleichmäßig über den Pfad verteilen.

Die Anzahl der Punkt kannst du frei wählen. Nimmst du eine sehr hohe Zahl, dann dauert es länger, bis du alle Punkte miteinander verglichen hast. Wählst du aber eine zu niedrige Zahl, dann verlieren deine Zeichnungen irgendwann die Merkmale, die sie unterscheidbar machen.

a) Teste für mehrere Zeichen, wie viele Punkte du benötigst, damit du sie noch erkennen kannst. Versuche Zeichen zu finden, bei denen diese Anzahl an Punkten sich unterscheidet:

Zeichen	**Zeichen mit 10 Punkten**	**Anzahl an Punkten, bei der Zeichen noch erkennbar ist**

Eine Entwicklung von OFFIS e.V. in Kooperation mit der Universität Oldenburg
im Auftrag der Wissensfabrik – Unternehmen für Deutschland e.V.

5.2 Den Unterschied zwischen zwei Pfaden messen

Jetzt brauchst du noch einen zweiten Pfad, mit dem du deine Zeichnung (deinen Pfad) vergleichen kannst. Dazu machst du eine zweite Variable und nennst sie „Beispiel“.

a) Erstelle einen zweiten Pfad:

   i. Erstelle eine zweite Variable „Beispiel“.

   ii. Weise dieser Variable eine Zeichnung zu.

   iii. Stelle mit „resample“ die Anzahl der Punkte ein.

b) Nachdem du ein Beispiel gespeichert hast, kannst du jetzt verschiedene andere Formen zeichnen, und mit dem “Unterschied” Block messen, wie sehr sie sich von dem Beispiel unterscheiden. Achte darauf, dass beide Pfade, Beispiel und neue Zeichnung, die gleiche Anzahl von Punkten haben. Du kannst dir wieder ein Mini-Programm basteln, das diesen Vergleich für dich vornimmt.

c) Baue dieses Skript nach und teste es mit sehr unterschiedlichen und sehr ähnlichen Gesten:

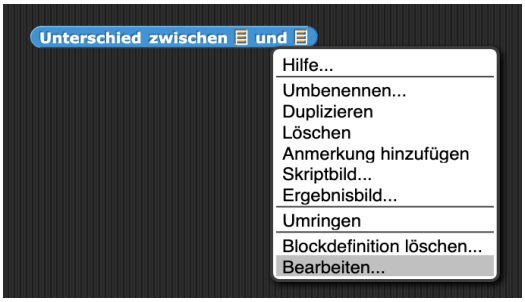
Ähnlichere Gesten:



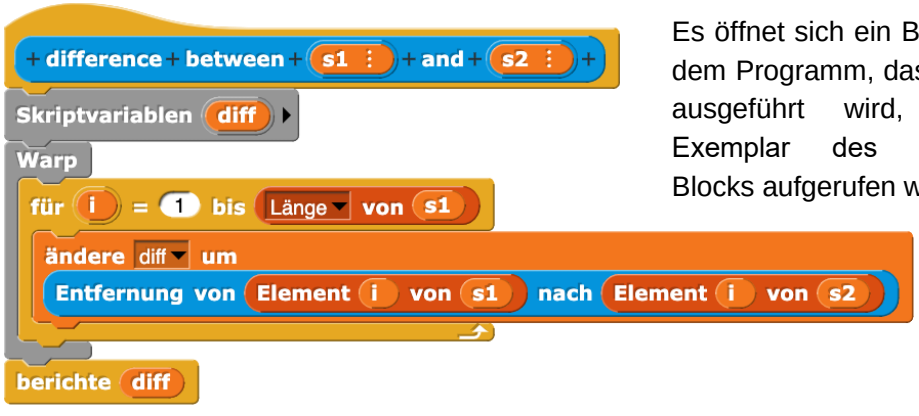
Sehr unterschiedliche Gesten:



d) Finde heraus wie der „Unterschied“-Block funktioniert. Sieh dir das folgende Programm an und versuche nachzuvollziehen was es macht, und beschreibe die Funktionsweise in eigenen Worten.



Klicke dafür mit der rechten Maustaste auf den Block und wähle „Bearbeiten…“. Auf einem Touchscreen bleibe einfach lange mit dem Finger auf dem Block, um das Kontextmenü zu öffnen.



Es öffnet sich ein Blockeditor mit dem Programm, das immer dann ausgeführt wird, wenn ein Exemplar des “Unterschied” Blocks aufgerufen wird.

Eine Entwicklung von OFFIS e.V. in Kooperation mit der Universität Oldenburg
im Auftrag der Wissensfabrik – Unternehmen für Deutschland e.V.

5.3 Beispiele für Gesten speichern

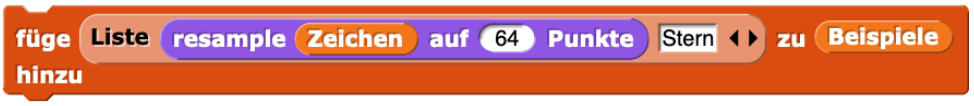
Damit dein Programm herausfinden kann, welchem Beispiel eine Zeichnung am nächsten ist, braucht diese mehrere Beispiele.

a) Erstelle also nochmal eine neue Variable und nenne sie “Beispiele” (Plural). Danach kannst du die Variable mit dem Namen “Beispiel” (Singular) löschen, wenn du willst, du brauchst sie nicht mehr.

   Die “Beispiele” Variable soll beliebig viele Pfade speichern können, deshalb initialisierst du sie als Liste. Das geht genauso, wie du es schon für die andere Variable mit dem Namen “Zeichen” getan hast:



b) Nun kannst du in Beispiele Beispielgesten speichern und mit Namen versehen. Male ein Zeichen und speichere es mit dem folgenden Skript:



   Dieser Teil des Programms macht eine neue Liste aus zwei Elementen, dem auf 64 Punkte normalisierten Pfad deiner aktuellen Zeichnung und einem Wort als Bezeichnung für den Pfad. Diese Liste wird den Beispielen hinzugefügt. Einer Liste weitere Listen hinzuzufügen kennst du schon, es ist das Gleiche, wie zu einem Pfad eine neue Liste aus x- und y-Koordinate hinzuzufügen.

c) Speichere für die Gesten, für die du dir schon eine Animation überlegt hast, in der „Beispiele“ – Liste einen Pfad und Namen ab (Siehe KI-A3.1.3; KI-A3.1.5 S. 5).

5.4 Die ähnlichste Geste ermitteln

In diesem Abschnitt baust du deinen eigenen Block, der beliebige Gesten erkennen kann. Alle dafür erforderlichen Bausteine hast du schon. Der Block soll ein Reporterblock sein, also eine Funktion mit zwei Eingaben:



Die erste Eingabe ist der Pfad, der zuletzt gezeichnet worden ist, der mit den Beispielen verglichen werden soll. Hierfür kann folgender Block verwendet werden:



Dieser “erkenne” Block soll ein Wort berichten, den Namen eines Beispiels, das der Zeichnung am ähnlichsten ist. Dieses Wort möchtest du an alle Objekte senden, um später ein Ereignis auszulösen



Eine Entwicklung von OFFIS e.V. in Kooperation mit der Universität Oldenburg im Auftrag der Wissensfabrik – Unternehmen für Deutschland e.V.

Nun bist du an der Reihe:

a) Erstelle jetzt den deinen eigenen „erkenne“ Block, indem du in einer beliebigen Kategorie auf das Plus-Zeichen am oberen Ende klickst. Wähle die passende Form (oval, d.h. Funktion), Farbe (hellblau für Fühlen), Geltungsbereich (“für alle”) und gib einen Namen ein.

b) Füge danach die beiden Eingaben hinzu, die du z.B. “Pfad” und “Muster” nennen, und als Typ “Liste” markieren kannst. Du kannst neue Eingabefelder oder Beschreibungsfelder für einen Block erstellen, indem du auf ein Pluszeichen an der Stelle klickst, an der du die Eingabe haben möchtest. Das Ergebnis könnte bspw. wie folgt aussehen:



c) Nachdem du den obigen Block zusammengeklickt hast, überlege dir in deinen eigenen Worten, was passieren soll, wenn der Block aufgerufen wird und wie du das Beispiel ermitteln kannst, das einer Zeichnung am nächsten kommt!

d) Nun baue den Block tatsächlich zusammen. Dafür benötigst du die folgenden Bausteine:



Du kannst den „Skriptvariablen“ Block mit den schwarzen Pfeilen um neue Variablen erweitern.

Du kannst Skriptvariablen und Blockvariablen wie im „für jedes Element von“ Block umbenennen, indem du einmal auf die jeweilige orange Variable klickst.

Zuerst bestimmst du diejenigen Variablen, die deine Funktion braucht:

- Unterschied von zwei Pfaden
- bisher kleinster Unterschied
- Beispiel mit bisher kleinstem Unterschied (Treffer)

Hinweis: weitere Erklärungen findest du auf KI-B3.1.5, S. 7

e) Verbinde dieses Skript, also deinen „erkenne“-Block mit dem Zeichenprogramm, das den Malstift steuert.

Eine Entwicklung von OFFIS e.V. in Kooperation mit der Universität Oldenburg im Auftrag der Wissensfabrik – Unternehmen für Deutschland e.V.

6. Eine Geschichte erzählen

In diesem Teil sollen die erkannten Zeichnungen mit den Animationen verbunden werden, um eine Geschichte zu erzählen.

6.1 Auf Nachrichten reagieren

Dein Zeichenskript sendet eine Nachricht an alle Objekte, sobald eine Zeichnung erkannt wurde. Auf diese Nachricht können andere Skripte reagieren.



Dieser Startblock löst das Skript aus, mit dem er verbunden ist, wenn er die Nachricht empfängt, die im Eingabefeld angegeben ist.

a) Erstelle eine Nachricht, auf die der Startblock reagieren soll, indem du auf den kleinen schwarzen Pfeil im Eingabefeld klickst und „Neu…“ auswählst. Gib eine Nachricht im Fenster ein und klicke „OK“.

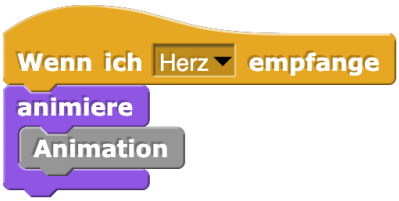
Wähle als Nachricht ein Beispiel aus deiner „Beispiele“-Liste aus, z.B. „Herz“.







b) Verbinde den Startblock für eine entsprechende Nachricht mit der passenden Animation für den Pfad, der deiner aktuellen Zeichnung am ähnlichsten ist:



6.2 Abschlussprojekt

Erstelle ein Abschlussprojekt, mit dem du eine kurze Geschichte illustrieren kannst. Zeichne Gesten, während du die Geschichte erzählst, und animiere diese, um deine Worte zu unterstreichen.

Übrigens kannst du mit den Nachrichten auch Skripte in anderen Objekten oder den Hintergrund steuern, um dein Projekt noch auszuschmücken.

Eine Entwicklung von OFFIS e.V. in Kooperation mit der Universität Oldenburg
im Auftrag der Wissensfabrik – Unternehmen für Deutschland e.V.